

TOOL: How Effective Are LLM-Based Approaches in Repairing Overflow Vulnerabilities

Anonymous Author(s)*

ABSTRACT

1 INTRODUCTION

Overflow vulnerabilities are a class of software weaknesses that occur when a program writes more data to a buffer than it can hold, potentially leading to serious consequences such as arbitrary code execution or memory corruption. These vulnerabilities are particularly common in low-level programming languages like C and C++, where manual memory management is widely used. Based on our analysis of two widely used vulnerability repair datasets—BigVul[3] and CVEFixes[2]—we observe that overflow-related issues account for a significant proportion of real-world vulnerabilities: 2,054 out of 4,067 CVEs (50.5%) and 7,865 out of 14,216 vulnerable functions (55.3%) are linked to overflow-related CWE entries. Furthermore, according to the 2024 CWE Top 25 Most Dangerous Software Weaknesses[10], six out of the top 25 entries fall into the category of overflow vulnerabilities, highlighting their continued prevalence and severity in modern software systems.

llm-based vulnerability repair refers to the use of large language models (LLMs) to automatically detect and fix software vulnerabilities. LLMs, trained on vast amounts of code, can understand programming languages and generate code snippets that address specific vulnerabilities. This approach leverages the capabilities of LLMs to analyze code patterns and suggest fixes, potentially improving the efficiency and effectiveness of vulnerability repair processes.

2 EMPIRICAL STUDY

2.1 Data Collection

We collect our dataset through a two-step process. First, we identify CWE entries associated with overflow vulnerabilities. Then, we extract relevant vulnerability samples from widely-used vulnerability repair datasets based on these identified CWEs.

To systematically identify overflow-related vulnerabilities, we adopt a two-pronged approach to collecting relevant CWE entries. First, we refer to the Common Weakness Enumeration (CWE) framework[12], specifically utilizing CWE View-1000[13], also known as the “Research Concepts View.” This view provides a semantically organized structure of software weaknesses tailored for academic analysis, emphasizing conceptual clarity and hierarchy. Based on this taxonomy, we identify CWE-119 — Improper Restriction of Operations within the Bounds of a Memory Buffer — and its descendant nodes as representative of overflow vulnerabilities. The complete list of selected CWE entries is provided in Table 1. Second, to complement the taxonomy-driven selection, we conduct a survey of recent research on vulnerability detection and repair. Using keywords such as “overflow,” “detect,” and “repair,” we search for papers published in top-tier software engineering and

security venues (e.g., CCS¹, ASE²). From these works[1, 4–9, 11, 14], we analyze the datasets used and extract the CWE identifiers associated with overflow vulnerabilities. We find that most of these CWEs are already included in the CWE View-1000 hierarchy; for those not covered, we incorporate them into our list to ensure comprehensive coverage and consistency with real-world research practices.

Next, we extract overflow-related vulnerability samples from two widely used vulnerability repair datasets: BigVul[3] and CVEFixes[2]. BigVul is a large-scale dataset that collects real-world vulnerabilities and corresponding fixes from open-source projects hosted on GitHub. CVEFixes, on the other hand, focuses on Common Vulnerabilities and Exposures (CVEs) and provides fine-grained fix commits associated with specific CVE entries. To maintain consistency with BigVul, we only retain C/C++ vulnerability samples from CVEFixes. We merge the two datasets following the standard practice established in prior work[15], including deduplication and partitioning at the function level. Before filtering, the merged dataset consists of XXX CWEs, XXX CVEs, and XXX vulnerable functions. By applying the set of CWE identifiers identified in the previous step, we construct a refined dataset focused on overflow vulnerabilities, which includes XXX distinct CVEs and XXX vulnerable functions. A detailed breakdown of the final dataset is presented in Table 2.

In addition, we analyze the distribution of the number of vulnerable functions associated with each CVE, as shown in Table 3. The results indicate that XXX CVEs contain only a single vulnerable function, accounting for XXX%, while XXX CVEs involve multiple vulnerable functions, accounting for XXX%. Existing studies have largely focused on repairing single-function vulnerabilities, often simplifying multi-function vulnerabilities into single-function cases for processing. However, repairing multi-function vulnerabilities is inherently more challenging: when reduced to the single-function level, large language models lack sufficient contextual information, making correct repairs infeasible. Therefore, directly simplifying multi-function vulnerabilities into single-function cases is problematic. In this work, we further analyze, through an empirical study, the impact of these unrepairable multi-function vulnerabilities on overall repair performance.

¹ACM Conference on Computer and Communications Security

²IEEE/ACM International Conference on Automated Software Engineering

ID	Name
119	Improper Restriction of Operations within the Bounds of a Memory Buffer
120	Buffer Copy without Checking Size of Input ('Classic Buffer Overflow')
121	Stack-based Buffer Overflow
122	Heap-based Buffer Overflow
123	Write-what-where Condition
124	Buffer Underwrite (Buffer Underflow)
125	Out-of-bounds Read
126	Buffer Over-read
127	Buffer Under-read
190*	Integer Overflow or Wraparound
415	Double Free
416	Use After Free
466	Return of Pointer Value Outside of Expected Range
476*	NULL Pointer Dereference
761*	Free of Pointer not at Start of Buffer
786	Access of Memory Location Before Start of Buffer
787	Out-of-bounds Write
788	Access of Memory Location After End of Buffer
805	Buffer Access with Incorrect Length Value
806	Buffer Access Using Size of Source Buffer
822	Untrusted Pointer Dereference
823	Use of Out-of-range Pointer Offset
824	Access of Uninitialized Pointer
825	Expired Pointer Dereference

Table 1: List of CWE entries related to overflow vulnerabilities. Entries marked with * are added based on recent literature.

CWE ID	CVE Count	Vulnerability Count
CWE-119	483	1532
CWE-120	71	249
CWE-121	9	14
CWE-122	47	69
CWE-125	425	1417
CWE-126	2	2
CWE-190	159	2539
CWE-415	42	162
CWE-416	215	463
CWE-476	267	456
CWE-787	319	943
CWE-823	4	5
CWE-824	11	14
Total	2054	7865

Table 2: CWE Distribution

2.2 RQ design

- **RQ1: Vulnerability Repairability.** Are all vulnerable functions in the dataset repairable?
- **RQ2. Repairable Vulnerability Characterization.** What are the key characteristics of overflow vulnerabilities?

Number of Functions	Number of CVEs
1	xxx
2	xxx
3	xxx
4	xxx
5	xxx
6	xxx
7	xxx
8	xxx
9	xxx
10	xxx
>10	xxx
Total	xxx

Table 3: Distribution of CVEs by Number of Vulnerable Functions(to be later replaced with a bar chart)

- **Fixing Strategies.** What kinds of repair strategies are commonly applied to overflow vulnerabilities?
- **RQ3 Evaluation of Existing LLM-Based Vulnerability Repair Approaches** Assessing the effectiveness of these methods in terms of (i) function repairability, (ii) vulnerability type, and (iii) applied repair strategies.

2.3 RQ1: Vulnerability Repairability

Based on prior research, a multi-function vulnerability may include functions that are unrelated to the actual repair. These functions often remain logically equivalent before and after the fix, serving mainly to improve code readability rather than contributing to the repair itself. Among the functions that are directly related to the repair, we further distinguish between two categories: functions that are repairable from a single-function perspective and those that are not. The former refers to cases where the vulnerability trigger logic can be inferred solely from the code of the function, enabling effective repair. The latter, by contrast, refers to cases where the vulnerability trigger logic cannot be deduced from the function alone, making it impossible to repair in isolation. Clearly, functions that are unrepairable from a single-function perspective necessarily depend on contextual information provided by other functions to identify the vulnerability trigger logic and guide the repair.

Both irrelevant functions and single-function-unrepairable functions are unique challenges associated with multi-function vulnerabilities. In contrast, functions corresponding to single-function vulnerabilities are always directly related to the vulnerability and thus do not exhibit these issues. The relationships are summarized in Table 4.

To analyze this phenomenon, we performed stratified sampling on our dataset, ensuring that the distribution of CWEs and the distribution of the number of functions per CVE were consistent with those of the overall dataset. At a 99% confidence level with a 5% margin of error, we randomly selected **xx** CVEs, comprising **xxx** functions, as our study sample. Through manual inspection, we found that among the functions associated with multi-function vulnerabilities, **xxx** were unrelated to the vulnerability repair, **xxx** were unrepairable from a single-function perspective, and **xxx** were repairable from a single-function perspective. The detailed breakdown is presented in Table 5.

Vulnerability Type	IRF	UR (SFP)	R (SFP)
Single-Function Vulnerability	N	N	Y
Multi-Function Vulnerability	Y	Y	Y

Table 4: Classification of Vulnerability-Related Functions. IRF = Irrelevant Repair Functions; UR (SFP) = Unrepairable (Single-Function Perspective); R (SFP) = Repairable (Single-Function Perspective).

Single-Function Vulnerability	Multi-Function Vulnerability		
	IRF	UR (SFP)	R (SFP)
xxx	xxx	xxx	xxx

Table 5: Number of Functions by Vulnerability Type and Repairability Category. IRF = Irrelevant Repair Functions; UR (SFP) = Unrepairable (Single-Function Perspective); R (SFP) = Repairable (Single-Function Perspective).

For irrelevant repair functions, we exclude them from the validation set used to evaluate vulnerability-repair tools. By contrast, functions that are unrepairable when viewed in isolation are treated as an intrinsic characteristic of multi-function vulnerabilities and therefore cannot be reduced to single-function cases.

We further investigated the underlying causes of these irrelevant repair functions. Such cases may include situations where the pre- and post-repair functions are semantically equivalent and the changes merely improve code readability, or where additional functionality is introduced that is unrelated to the vulnerability fix. Functions that remain unrepairable under a single-function setting often do so because their remediation requires coordinated modifications across multiple functions, employs macro definitions, or depends on variables and contextual information that cannot be inferred from the function alone. As shown in the Table 6.

Category	Description
ccc	ccc

Table 6: Categories of Irrelevant Repair Functions and Functions Unrepairable from a Single-Function Perspective.

To explore whether these unrepairable functions could become repairable with broader context, we examined the effect of incorporating successfully repaired functions as supplementary information. Our manual analysis shows that, among the sampled xxx functions unrepairable from a single-function perspective, xxx became repairable when additional successfully repaired functions were provided as context, representing xxx%. This finding indicates that such functions rely on inter-function contextual cues to expose the vulnerability-triggering logic and thereby guide the repair process.

Answer to RQ1

In multi-function vulnerabilities, approximately xxx% of the functions are irrelevant repair functions and about xxx% are unrepairable from a single-function perspective. Neither category can be simplified into single-function vulnerabilities. The former are excluded from the validation set for evaluating vulnerability repair tools, while the latter can become repairable in xx% of cases when contextual information from already repaired functions is incorporated.

2.4 RQ2. Repairable Vulnerability Characterization

For RQ1, we focus on vulnerabilities that can be directly repaired, including functions from single-function vulnerabilities and functions that are repairable from a single-function perspective within multi-function vulnerabilities. We further analyze the repair logic of these vulnerable functions. Specifically, we manually examined and categorized the repair logic of xxx vulnerable functions, with the detailed classification presented in Figure 1.

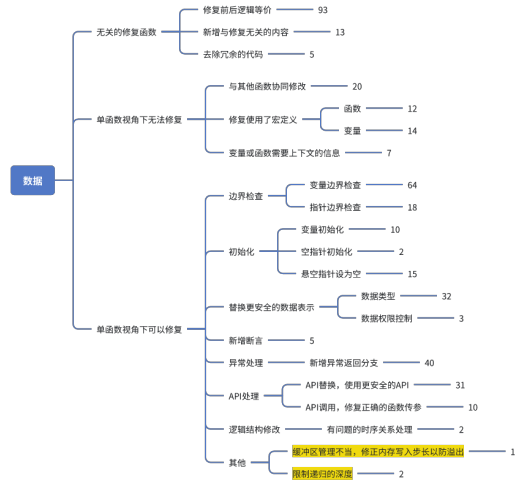


Figure 1: Classification of repair logic for vulnerabilities that can be repaired in isolation, including functions from single-function vulnerabilities and single-function-repairable functions in multi-function vulnerabilities.

This section is to be completed later and should include specific classification diagrams and quantitative data displays.

Answer to RQ2

2.5 RQ3: Evaluation of Existing LLM-Based Vulnerability Repair Approaches

2.5.1 Setup. The evaluation is conducted using several baselines, including simple prompting and agent-based approaches. Relevant prior work will be reviewed to position our methodology.

2.5.2 *Statistical Analysis.* We perform statistical analysis using common evaluation metrics such as CodeBLEU, Exact Match, and other relevant measures.

2.5.3 *Manual Sampling Analysis.* Through manual inspection, we expect to observe that multi-agent approaches outperform single-agent ones, while also identifying potential issues of over-repair.

Answer to RQ3

RQ3 aims to evaluate the effectiveness of existing approaches with respect to (i) function repairability, (ii) vulnerability type, and (iii) applied repair strategies, using a combination of statistical metrics and manual sampling analysis. The evaluation is expected to reveal three key issues: 1. Irrelevant repair functions and functions unrepairable from a single-function perspective significantly affect the results of prior methods. 2. Existing statistical metrics—such as CodeBLEU and Exact Match—do not adequately capture the performance of LLM-based vulnerability repair. 3. Numerous cases of over-repair are observed.

3 DESIGN OF TOOL

In this section, we present the design of TOOL.

- A. Filter irrelevant functions
- B. Classify Intra-Function Repairable and Intra-Function UnRepairable functions.
- C. Repair Intra-Function Repairable
- D. Repair Intra-Function UnRepairable using "Repaired Intra-Function Repairable Functions"

4 EVALUATION

We evaluate TOOL based on the following research questions.

- **RQ1. Effectiveness Evaluation.** How is the effectiveness of TOOL compared to the state-of-the-art tools?
- **RQ2. Ablation Study.** How does each component contribute to TOOL's effectiveness?

4.1 Setup

4.2 Setup

Dataset.

Baselines.

Metrics.

4.3 Discussion

Threats. First,

Limitations. First,

5 RELATED WORK

We review previous work related to

6 CONCLUSIONS

In this paper,

REFERENCES

- [1] Jinsheng Ba, Gregory J Duck, and Abhik Roychoudhury. 2022. Efficient greybox fuzzing to detect memory errors. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*. 1–12.
- [2] Guru Bhandari, Amara Naseer, and Leon Moonen. 2021. CVEfixes: Automated Collection of Vulnerabilities and Their Fixes from Open-Source Software. In *Proceedings of the 17th International Conference on Predictive Models and Data Analytics in Software Engineering (PROMISE '21)*. ACM, 10. <https://doi.org/10.1145/3475960.3475985>
- [3] Jiahao Fan, Yi Li, Shaohua Wang, and Tien N. Nguyen. 2020. A C/C++ Code Vulnerability Dataset with Code Changes and CVE Summaries. In *Proceedings of the 17th International Conference on Mining Software Repositories* (Seoul, Republic of Korea) (MSR '20). Association for Computing Machinery, New York, NY, USA, 508–512. <https://doi.org/10.1145/3379597.3387501>
- [4] Floris Gorter, Taddeus Kroes, Herbert Bos, and Cristiano Giuffrida. 2024. Sticky tags: Efficient and deterministic spatial memory error mitigation using persistent memory tags. In *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE, 4239–4257.
- [5] Yiyuan Guo, Peisen Yao, and Charles Zhang. 2024. Precise Compositional Buffer Overflow Detection via Heap Disjointness. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 63–75.
- [6] Wooseok Kang, Byoungso Son, and Kihong Heo. 2022. Tracer: Signature-based static analysis for detecting recurring vulnerabilities. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*. 1695–1708.
- [7] Tan Khang Le, Saba Alimadadi, and Steven Y. Ko. 2024. A Study of Vulnerability Repair in JavaScript Programs with Large Language Models. In *Companion Proceedings of the ACM Web Conference 2024* (Singapore, Singapore) (WWW '24). Association for Computing Machinery, New York, NY, USA, 666–669. <https://doi.org/10.1145/3589335.3651463>
- [8] Yuan Li, Wende Tan, Zhizheng Lv, Songtao Yang, Mathias Payer, Ying Liu, and Chao Zhang. 2022. Pacmem: Enforcing spatial and temporal memory safety via arm pointer authentication. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*. 1901–1915.
- [9] Zhanpeng Liu, Yi Rong, Chenyang Li, Wende Tan, Yuan Li, Xinhui Han, Songtao Yang, and Chao Zhang. 2025. CCTAG: Configurable and Combinable Tagged Architecture. In *NDSS*.
- [10] MITRE. 2024. CWE Top 25 Most Dangerous Software Weaknesses for 2024. <https://cwe.mitre.org/top25/>. Accessed: 2025-07-21.
- [11] Benjamin Steenhoeck, Md Mahbubur Rahman, Richard Jiles, and Wei Le. 2023. An empirical study of deep learning models for vulnerability detection. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2237–2248.
- [12] The MITRE Corporation. 2024. *Common Weakness Enumeration (CWE)*. MITRE. <https://cwe.mitre.org/> Accessed: 2025-07-21.
- [13] The MITRE Corporation. 2024. *Research Concepts (View-1000) – Common Weakness Enumeration (CWE) Detail for CWE-1000*. MITRE. <https://cwe.mitre.org/data/definitions/1000.html> Accessed: 2025-07-21.
- [14] Junwei Zhang, Zhongxin Liu, Xing Hu, Xin Xia, and Shanping Li. 2023. Vulnerability detection by learning from syntax-based execution paths of code. *IEEE Transactions on Software Engineering* 49, 8 (2023), 4196–4212.
- [15] Xin Zhou, Kisub Kim, Bowen Xu, DongGyun Han, and David Lo. 2024. Large Language Model as Synthesizer: Fusing Diverse Inputs for Better Automatic Vulnerability Repair. *CoRR* (2024).